



l'école d'ingénierie
informatique

FutureKawa

Solution de suivi des stocks et
des conditions de stockage (IoT)

Dossier technique

MSPR TPRE814 — Bloc 4 — RNCP 35584

Membres du groupe

Nom Prénom	Rôle
Tarek IZERROUKEN	Chef de Projet
Sofiane CHELGHOUM	Responsable IoT
Yassine MOUDJAHID	Développement back-end
Kelly TSANGA	Développement back-end
Sara HADDAD	Développement front-end

Encadrant : Samah GHALLOUSSI
Établissement : EPSI

Table des matières

I	Présentation du projet	4
1	Introduction	5
1.1	Contexte métier	5
1.2	Problématique et enjeux	5
1.3	Objectifs du projet	5
1.4	Périmètre et démarche	6
1.5	Organisation du document	6
2	Collecte des besoins métier	7
2.1	Préparation de l'interview	7
2.2	Questionnaire utilisé	7
2.3	Méthodologie de retranscription	8
2.4	Résultat de la collecte	8
II	Conception de la solution	9
3	Architecture globale	10
3.1	Vue d'ensemble	10
3.2	Composants et flux	10
3.2.1	Back-end pays	10
3.2.2	Back-end central	11
3.2.3	Front-end web	11
3.2.4	Synthèse des composants	11
3.2.5	Flux de données	11
3.3	API REST et points d'entrée	11
3.4	Modèle de données	12
3.5	Dispositif d'alertes	14
3.5.1	Seuils par pays	14
3.5.2	Logique de décision et anti-fausse-alerte	15
3.6	Justification des choix	16
3.7	Sécurité et gestion des accès	17
3.8	Robustesse et tolérance aux pannes	17
4	Conception du module IoT	19
4.1	Choix matériel	19
4.2	Câblage	19
4.3	Environnement logiciel	20
4.4	Protocole MQTT	20
4.4.1	Topics	20
4.4.2	Format du payload	21
4.4.3	Fréquence d'envoi	21

4.5	Configuration et pilotage à distance	21
4.6	Reconnexion et gestion des erreurs	21
4.7	Limites et risques	21
III	Qualité et industrialisation	23
5	Plans de tests détaillés	24
5.1	Stratégie de test	24
5.2	Cas de test	25
5.2.1	Back-end pays — moteur d’alertes	25
5.2.2	Back-end central — isolation et consolidation	25
5.2.3	Front-end — fonctions d’affichage utilisateur	26
5.3	Données de test	26
5.4	Critères de réussite	26
5.5	Gestion des anomalies	26
5.6	Limites et perspectives	26
6	Industrialisation : intégration et déploiement continu	27
6.1	Objectifs et démarche	27
6.2	Une chaîne reproductible	27
6.3	Le pipeline d’intégration continue	27
6.4	Packaging et construction des images	28
6.5	La livraison continue	28
6.6	Le déploiement Kubernetes	29
6.7	Preuves d’exécution	29
6.8	Limites et perspectives	30
IV	Déploiement, accompagnement et bilan	31
7	Documentation utilisateur	32
7.1	Objet et public visé	32
7.2	Connexion et principes généraux	32
7.3	Le tableau de bord	32
7.4	Consulter les stocks et les lots	33
7.5	Lire les mesures (courbes)	34
7.6	Comprendre et traiter les alertes	35
7.7	Fonctions d’administration	36
7.7.1	Gestion des utilisateurs	36
7.7.2	Rôles et permissions	36
7.7.3	Paramètres système	37
7.7.4	Journal d’audit	37
7.8	FAQ — résolution des problèmes courants	38
8	Conduite du changement	39
8.1	Justification de la démarche	39
8.2	Plan d’action sur les quatre axes	39
8.3	Mise en application	39

9	Conclusion	41
9.1	Bilan du travail réalisé	41
9.2	Conformité aux besoins	41
9.3	Limites	41
9.4	Perspectives	41

Première partie

Présentation du projet

Chapitre 1

Introduction

1.1 Contexte métier

FutureKawa est une entreprise internationale spécialisée dans la caféiculture et la logistique de *café vert* (non torréfié). Elle exploite des plantations réparties sur trois pays d’Amérique du Sud — le **Brésil**, l’**Équateur** et la **Colombie** — et couvre l’ensemble de la chaîne de valeur : production agricole, constitution de *lots* identifiés, **stockage en entrepôts**, puis distribution internationale en B2B (torréfacteurs, marques, importateurs).

Dans cette chaîne, le stockage est une étape critique : la qualité aromatique des grains dépend directement des **conditions de conservation** (température et humidité). Une dérive prolongée entraîne une altération, un déclassement du lot, voire une perte sèche.

1.2 Problématique et enjeux

Aujourd’hui, le suivi des conditions de stockage souffre de plusieurs limites :

- des conditions **variables d’un entrepôt à l’autre**, avec un impact direct sur la qualité ;
- un suivi encore **semi-manuel** (relevés ponctuels, tableurs, outils hétérogènes), difficilement auditable ;
- une logique de rotation **FIFO** (expédier en priorité les lots les plus anciens) dont la visibilité n’est pas fiable ;
- des clients qui exigent des **preuves** : historique des conditions de stockage et gestion proactive des non-conformités.

Ces irritants pèsent directement sur le taux de réclamation, les coûts de pertes et la valorisation des lots premium.

1.3 Objectifs du projet

Le projet, commandité par les Directions des Opérations et de la Qualité, vise à doter FutureKawa d’un **outil unique et fiable** permettant de :

- centraliser le suivi des stocks par pays et par entrepôt ;
- garantir la **traçabilité** des lots depuis leur entrée en stockage ;
- surveiller les conditions de conservation via des **relevés IoT automatisés** (température / humidité) ;
- **détecter et signaler** les situations à risque : dérives de conditions et lots trop anciens ;
- poser une base technique préparant la *phase 2* : l’automatisation des équipements d’entrepôt (chauffage, humidification, aération).

1.4 Périmètre et démarche

La solution est livrée sous forme de **prototype avancé**, destiné à démontrer la faisabilité et à servir de socle à une industrialisation. Elle repose sur une **architecture distribuée pays ↔ siège** :

- un **back-end pays** conteneurisé (API REST, base SQL, broker MQTT local, dispositif d'alertes par e-mail), instancié pour les trois pays ;
- un **back-end central** au siège, qui interroge les pays et consolide les données ;
- un **front-end web** de consultation (stocks, courbes, alertes) ;
- un **module IoT** (microcontrôleur + capteur température / humidité) publiant ses mesures via MQTT.

Le projet a été mené avec une exigence de qualité « projet » : tests automatisés, intégration continue (CI/CD) et documentation.

1.5 Organisation du document

Ce dossier suit le cycle du projet, de l'expression du besoin jusqu'au bilan, et s'organise en quatre parties :

- la **première partie** recueille les **besoins métier** auprès des parties prenantes et en dégage les fonctionnalités prioritaires ;
- la **deuxième partie** présente la **conception de la solution** : l'**architecture globale** (chapitre 3 : composants, flux, API, modèle de données, alertes, sécurité, robustesse) et la **conception du module IoT** (chapitre 4 : matériel, câblage, protocole MQTT, gestion des erreurs) ;
- la **troisième partie** traite de la **qualité et de l'industrialisation** : les **plans de tests** (chapitre 5) et la chaîne d'**intégration et de déploiement continu** (chapitre 6) ;
- la **quatrième partie** couvre le **déploiement, l'accompagnement et le bilan** : la **documentation utilisateur**, la **conduite du changement** et la **conclusion** du projet.

Chapitre 2

Collecte des besoins métier

2.1 Préparation de l'interview

Afin d'identifier les besoins réels des utilisateurs de la solution FutureKawa, une phase de collecte des besoins a été menée sous forme d'**entretiens semi-directifs** avec les parties prenantes du projet : responsables d'exploitation et d'entrepôt, référents qualité, équipes Supply Chain et Direction SI. L'objectif était de comprendre leurs activités quotidiennes, leurs contraintes de terrain et leurs attentes fonctionnelles vis-à-vis de la future solution de suivi des stocks et des conditions de stockage.

La méthodologie retenue repose sur un questionnaire structuré en **quatre catégories** : le contexte d'utilisation actuel, les problématiques rencontrées, les fonctionnalités souhaitées et les contraintes techniques ou organisationnelles à respecter.

2.2 Questionnaire utilisé

A. Contexte et activité actuelle

- Comment suivez-vous aujourd'hui les conditions de stockage (température, humidité) dans vos entrepôts ?
- Quels outils utilisez-vous actuellement (relevés papier, tableurs, autre logiciel) ?
- Qui sont les utilisateurs concernés (exploitation, entrepôt, qualité, siège) ?

B. Problématiques rencontrées

- Quelles difficultés rencontrez-vous avec le suivi manuel actuel des conditions ?
- Comment détectez-vous aujourd'hui une dérive des conditions ou un lot trop ancien ?
- Y a-t-il des pertes ou des réclamations liées à un défaut de traçabilité ?

C. Fonctionnalités attendues

- Quelles informations souhaitez-vous consulter (lots, mesures, historiques, alertes) ?
- Souhaitez-vous des alertes automatiques, et par quel canal (e-mail) ?
- Avez-vous besoin d'une vue consolidée multi-pays et de courbes température/humidité par lot ?

D. Contraintes techniques et organisationnelles

- L'accès doit-il être différencié selon le rôle et le pays de l'utilisateur ?
- Quelles contraintes de terrain (réseau variable, matériel limité) faut-il prendre en compte ?
- Quelles exigences de sécurité et de traçabilité s'appliquent aux données ?

2.3 Méthodologie de retranscription

Les réponses sont retranscrites sous forme de **tableau de synthèse**, permettant de croiser chaque besoin exprimé avec la fonctionnalité correspondante et son niveau de priorité.

Besoin exprimé	Fonctionnalité associée	Priorité
Centraliser le suivi des stocks par pays	Interface web centralisée	Haute
Automatiser les relevés de conditions	Capteurs IoT + broker MQTT	Haute
Être alerté des dérives et des péremptions	Moteur d'alertes + e-mail	Haute
Consulter l'historique et les courbes	Historisation des mesures	Moyenne
Consolider la vue multi-pays au siège	Back-end central d'agrégation	Moyenne
Sécuriser l'accès selon le rôle et le pays	Authentification JWT + RBAC	Haute

TABLE 2.1 – Retranscription des besoins et fonctionnalités associées.

2.4 Résultat de la collecte

Cette phase a permis de dégager les besoins fonctionnels prioritaires ayant guidé la conception de la solution : suivi centralisé des stocks, relevés IoT automatisés, alertes proactives (conditions et péremption), historisation des mesures et sécurité des accès par rôle et par pays. Ces besoins ont directement orienté les choix d'architecture applicative présentés dans la partie suivante.

Deuxième partie

Conception de la solution

Chapitre 3

Architecture globale

3.1 Vue d'ensemble

La solution adopte une **architecture distribuée** qui reflète l'organisation « groupe » de FutureKawa : chaque pays dispose de son propre système autonome, et un **siège central** consolide l'ensemble. Trois grands ensembles cohabitent :

- les **sites pays** (Brésil, Équateur, Colombie), chacun instrumenté par un module IoT et servi par un back-end local et sa base de données ;
- le **siège**, qui héberge le back-end central (consolidation) et le front-end web ;
- l'**infrastructure partagée** : le broker MQTT et le service d'envoi d'e-mails.

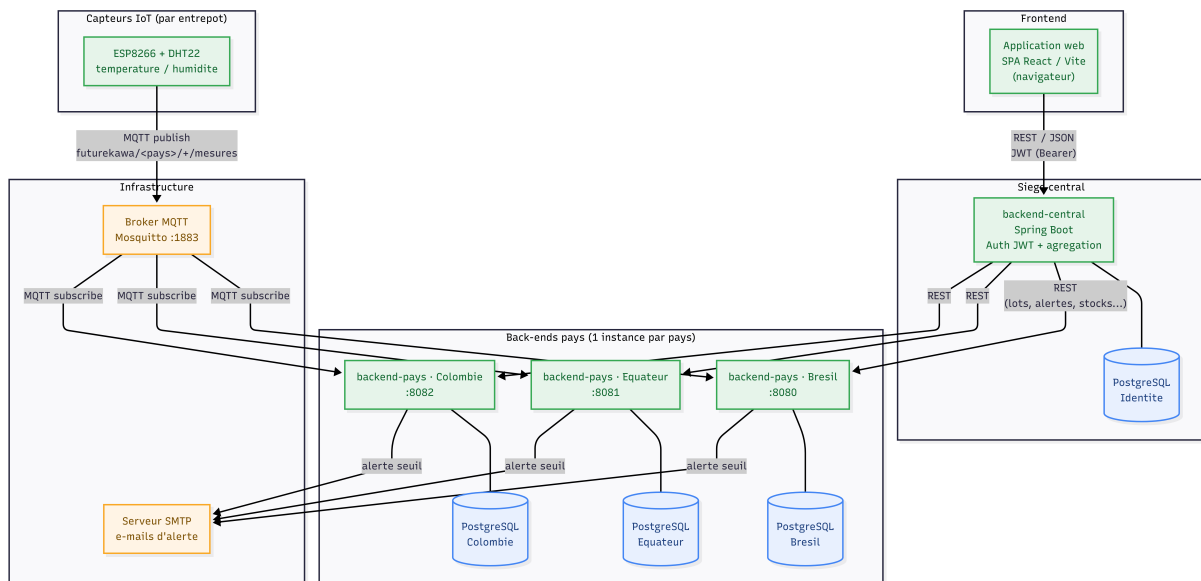


FIGURE 3.1 – Architecture globale : sites pays, siège central et flux (MQTT, REST).

3.2 Composants et flux

3.2.1 Back-end pays

Chaque pays est servi par une instance du **back-end pays** (Spring Boot, Java 21), déployée en trois exemplaires configurés par profil (Brésil, Équateur, Colombie). Il assure quatre rôles :

- **ingestion IoT** : il s'abonne au broker MQTT sur le topic 'futurekawa/<pays>/+/mesures' et persiste chaque mesure reçue ;
- **API REST** : gestion des lots et exposition des stocks, mesures et alertes vers le siège ;

- **moteur d’alertes** : évaluation des conditions à chaque mesure (cf. section 3.5) ;
- **notification** : envoi d’e-mails au responsable d’exploitation via SMTP.

Chaque instance possède sa **base PostgreSQL** dédiée et son **broker MQTT** local, conformément à l’exigence d’un back-end autonome par pays.

3.2.2 Back-end central

Le **back-end central** (Spring Boot) est hébergé au siège. Il n’héberge *aucune donnée métier* : il interroge les back-ends pays via des appels REST (client **PaysClient**) et **consolide** l’état des stocks, les mesures et les alertes pour alimenter le front-end. Il porte en revanche la **gestion des identités** (authentification, rôles et permissions), avec sa propre base « Identité » et une sécurité par **jeton JWT**.

3.2.3 Front-end web

Le **front-end** est une application web monopage (*SPA*) développée en **React**. Elle consomme uniquement l’API du back-end central, en s’authentifiant par jeton JWT (schéma *access token / refresh token*). Elle permet de sélectionner un pays et un lot, d’afficher la liste des lots triés par date de stockage (logique FIFO), de visualiser les **courbes de température et d’humidité**, et d’accéder aux alertes et aux statuts.

3.2.4 Synthèse des composants

Composant	Rôle	Technologie	Port
Back-end pays (×3)	Métier, IoT, alertes	Spring Boot / Java 21	8080–8082
Back-end central	Consolidation, auth	Spring Boot / Java 21	8090
Front-end	Interface web	React / Vite	—
Base de données	Persistance SQL	PostgreSQL	5432
Broker MQTT	Transport des mesures	Eclipse Mosquitto	1883

TABLE 3.1 – Composants de la solution et technologies retenues.

3.2.5 Flux de données

Deux flux principaux structurent le système :

1. **Flux montant (IoT → back-end)** : le capteur publie ses mesures en MQTT ; le broker les distribue ; le back-end pays, abonné, les persiste puis les évalue.
2. **Flux de consultation (front → central → pays)** : le front interroge le central en REST/JWT ; le central relaie et agrège les réponses des back-ends pays.

En cas d’alerte, un flux secondaire (**back-end pays → SMTP**) notifie par e-mail le responsable du pays concerné.

3.3 API REST et points d’entrée

La communication applicative repose sur des **API REST** échangeant du JSON. On distingue deux surfaces : l’API **interne** de chaque back-end pays (consommée par le central) et l’API **publique** du central (consommée par le front-end, après authentification). Ces interfaces sont documentées via **OpenAPI / Swagger**.

Méthode	Route	Description
GET	/exploitations	liste des exploitations
GET	/exploitations/{id}/entrepots	entrepôts d'une exploitation
GET	/entrepots/{id}/capteurs	capteurs d'un entrepôt
GET	/entrepots/{id}/mesures	mesures d'un entrepôt
POST	/lots	création d'un lot
GET	/lots	liste des lots (triés par date, FIFO)
GET	/lots/{id}	détail d'un lot
GET	/lots/{id}/mesures	mesures rattachées à un lot
GET	/alertes	liste des alertes

TABLE 3.2 – Principaux points d'entrée d'un back-end pays.

Méthode	Route	Description
POST	/auth/login	authentification, émission des jetons
POST	/auth/refresh	renouvellement du jeton d'accès
POST	/auth/logout	révocation du jeton de rafraîchissement
GET	/auth/me	profil de l'utilisateur courant
GET	/api/pays	pays et leur disponibilité
GET	/api/lots	lots consolidés (tous pays)
GET	/api/alertes	alertes consolidées
GET	/api/pays/{code}/lots	lots d'un pays
GET	/api/pays/{code}/alertes	alertes d'un pays
GET	/api/pays/{code}/lots/{lotId}/mesures	mesures d'un lot
GET	/api/pays/{code}/entrepots/{id}/mesures	mesures d'un entrepôt
GET	/users	gestion des utilisateurs
GET	/audit	consultation du journal d'audit

TABLE 3.3 – Principaux points d'entrée du back-end central (préfixe **/api** pour la consolidation).

La convention est classique : **GET** pour la consultation, **POST** pour la création. Le central expose une vue **consolidée** (routes **/api/...**) qui masque au front-end la répartition physique par pays.

3.4 Modèle de données

Cohérente avec le découpage distribué, la persistance repose sur **deux domaines de données distincts**, sans clé étrangère entre eux :

- la base « **Pays** » (métier IoT), dupliquée par pays : **Exploitation**, **Entrepot**, **Capteur**, **Mesure**, **Lot**, **Alerte** (figure 3.2) ;
- la base « **Identité** » (siège) : **Utilisateur**, **Role**, **Permission**, **RefreshToken**, **JournalAudit** (figure 3.3).

Le lien entre les deux domaines est **logique** (correspondance sur le champ **pays**) et non une contrainte physique, ce qui préserve la souveraineté des données de chaque pays.

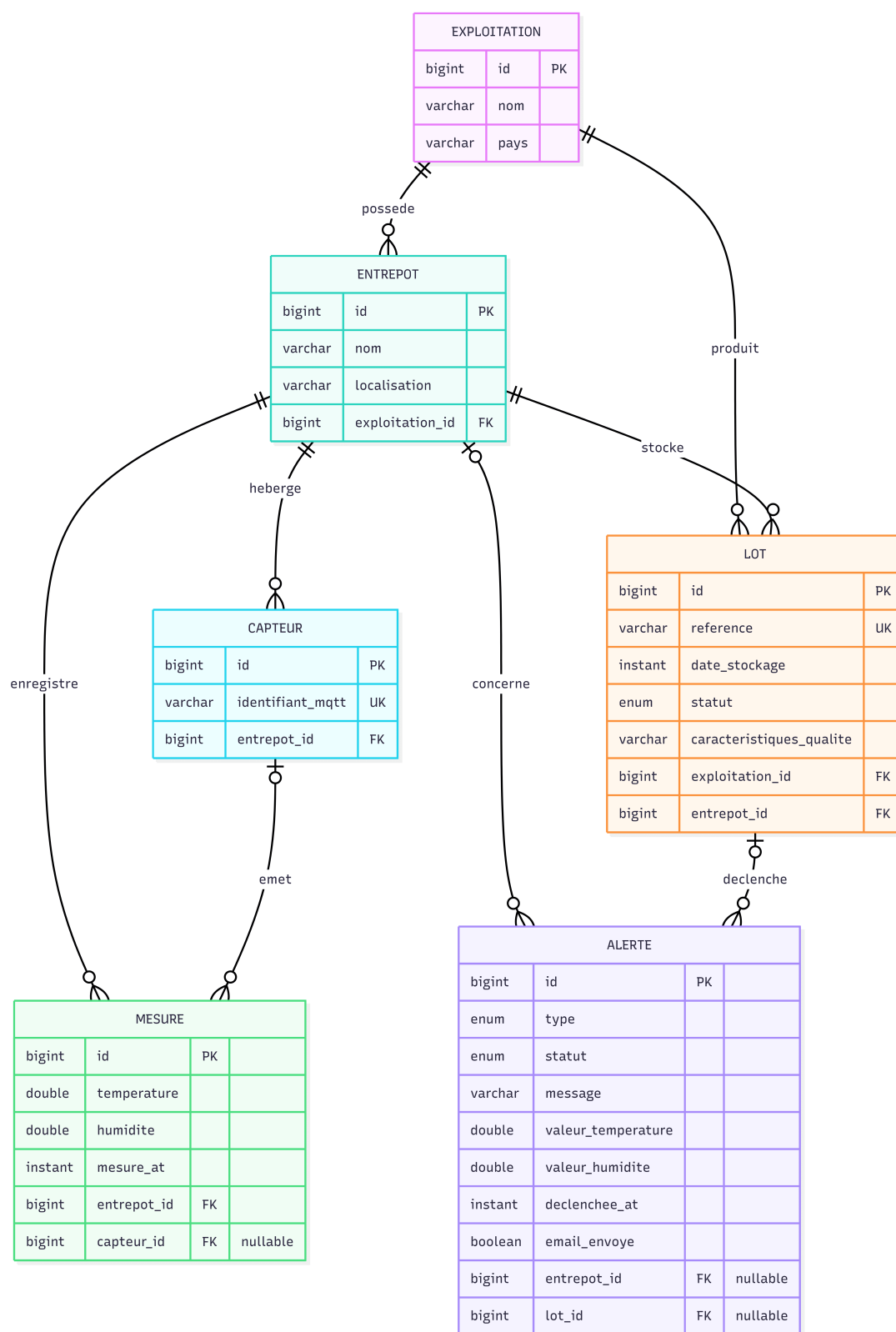


FIGURE 3.2 – Modèle de données du domaine « Pays » (métier IoT).

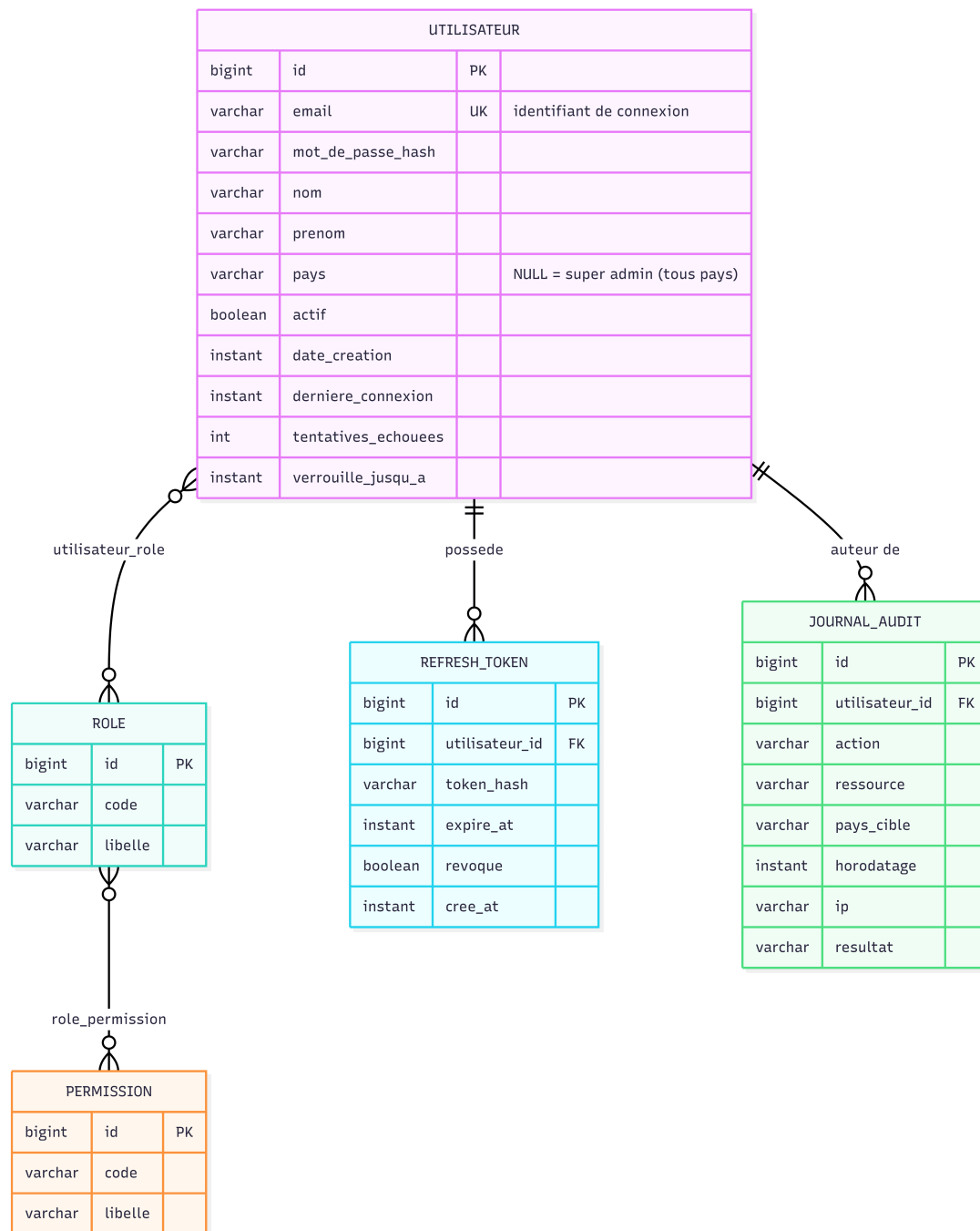


FIGURE 3.3 – Modèle de données du domaine « Identité » (authentification et habilitations).

3.5 Dispositif d'alertes

Le système lève une alerte automatique dans **deux cas** :

- **conditions hors plage** : température ou humidité en dehors de la bande acceptable pour le pays ;
- **péréemption** : lot dépassant **365 jours** de stockage.

3.5.1 Seuils par pays

Les conditions idéales dépendent du pays ; la tolérance est de $\pm 3^\circ\text{C}$ en température et de $\pm 2\%$ en humidité.

Pays	Temp. idéale	Humidité idéale	Tolérance
Brésil	29 °C	55 %	± 3 °C / ± 2 %
Équateur	31 °C	60 %	± 3 °C / ± 2 %
Colombie	26 °C	80 %	± 3 °C / ± 2 %

TABLE 3.4 – Conditions idéales et tolérances par pays (paramétrées par profil).

3.5.2 Logique de décision et anti-fausse-alerte

Pour éviter les alertes intempestives sur un pic transitoire, le moteur applique une **hystérésis** : un incident n'est ouvert qu'après **trois mesures consécutives** hors plage, et il est résolu automatiquement après **trois mesures consécutives** revenues dans la bande. Un mécanisme de **rappel** renotifye un incident toujours actif au plus toutes les 30 minutes (anti-spam). Les lots suivent les statuts CONFORME → EN_ALERTE → CONFORME, et les alertes les statuts ACTIVE → ACQUITTEE → RESOLUE.

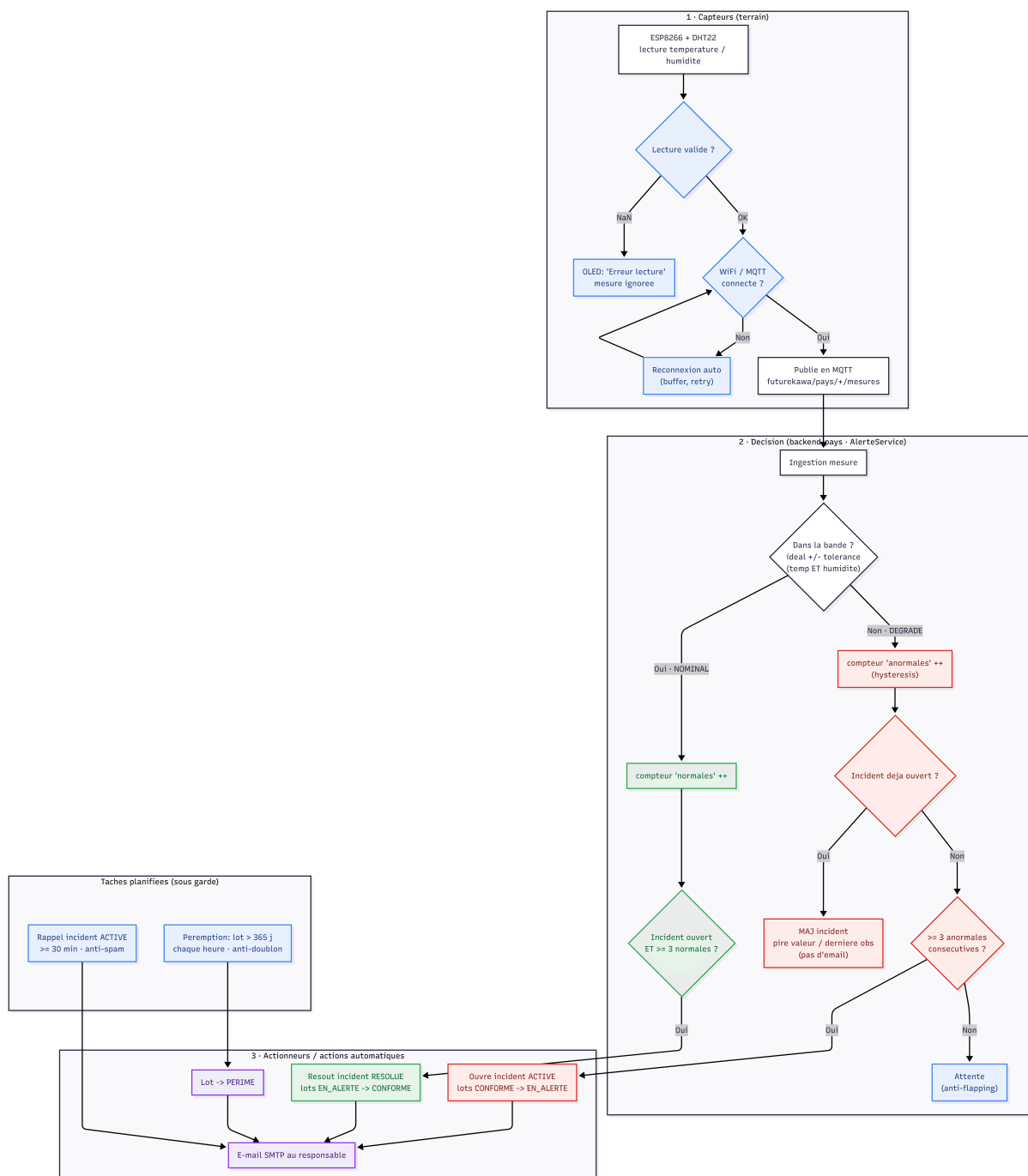


FIGURE 3.4 – Chaîne d’automatisation détaillée : capteurs → décision → actions.

3.6 Justification des choix

- **Un back-end et une base par pays** : garantit la *souveraineté des données*, isole les pannes (la défaillance d’un pays n’affecte pas les autres) et permet une montée en charge indépendante — au prix d’une consolidation applicative assurée par le central.
- **Un central sans base métier** : évite la duplication de données ; la source de vérité reste chaque pays, ce qui simplifie la cohérence.
- **MQTT pour l’IoT** : protocole *publish/subscribe* léger, adapté aux contraintes « terrain » (réseau variable, matériel limité). Il **découple** les capteurs des back-ends et tolère les déconnexions.

- **REST pour l’inter-service** : standard, interopérable et simple à superviser ; il structure le dialogue siège $\leftrightarrow$ pays.
- **JWT** : authentification *sans état*, adaptée à un système distribué.
- **PostgreSQL** : base SQL éprouvée, fiable pour la traçabilité exigée.
- **Spring Boot / React** : écosystèmes matures pour, respectivement, des back-ends métier robustes et une interface de *datavisualisation*.

3.7 Sécurité et gestion des accès

La sécurité est portée par le back-end central, qui centralise l’authentification et les habilitations.

Authentification par jeton (JWT). L’accès à l’API repose sur des **jetons JWT** : à la connexion, le central émet un **jeton d’accès** de courte durée (15 minutes) et un **jeton de rafraîchissement** révocable, stocké côté serveur. Les sessions sont **sans état** (*stateless*) : chaque requête porte son jeton, vérifié par un filtre dédié. Le jeton transporte l’e-mail de l’utilisateur, ses **rôles** et son **pays** d’affectation (* pour un accès à tous les pays).

Contrôle d’accès basé sur les rôles (RBAC). Les droits sont modélisés par des **rôles** porteurs de **permissions** (cf. modèle de données « Identité »). Un utilisateur hérite ainsi d’un ensemble de permissions selon ses rôles, ce qui permet une gestion fine et évolutive des accès.

Isolation par pays. Au-delà des rôles, un **cloisonnement par pays** est appliqué : un administrateur rattaché au Brésil ne peut consulter que les données brésiliennes, tandis qu’un super-administrateur (*) accède à l’ensemble. Toute tentative d’accès à un pays non autorisé est **refusée (403)** — comportement garanti par un test dédié (cf. chapitre 5).

Protection des comptes. Les mots de passe ne sont jamais stockés en clair (**empreinte** en base). Un mécanisme de **verrouillage** bloque temporairement un compte après plusieurs tentatives de connexion échouées, limitant les attaques par force brute.

Traçabilité. Les actions sensibles d’administration sont consignées dans un **journal d’audit** (auteur, action, ressource, pays cible, horodatage, résultat), au service de la traçabilité exigée par le métier.

3.8 Robustesse et tolérance aux pannes

Plusieurs mécanismes assurent la résilience de la solution :

- **Conteneurisation** : l’ensemble est reproductible via Docker Compose, et **orchestrable sur Kubernetes** (manifestes fournis) pour un déploiement multi-sites.
- **Auto-réparation** : sous Kubernetes, les services déclarent des sondes *liveness* et *readiness* ; un conteneur défaillant est automatiquement redémarré, et le back-end central est répliqué.
- **Isolation par pays** : le cloisonnement des back-ends et des bases confine les pannes à un seul site.
- **Découplage MQTT et reconnexion automatique** côté IoT : la perte temporaire du réseau n’interrompt pas durablement la collecte.
- **Anti-fausse-alerte** : l’hystérésis évite de déclencher des incidents sur des variations passagères.

- **Traçabilité** : journalisation applicative (logs) et journal d’audit des actions d’administration.
- **Industrialisation** : une chaîne d’intégration continue (CI/CD) rejoue automatiquement les tests et le packaging à chaque évolution (détaillée au titre du livrable dédié).

Chapitre 4

Conception du module IoT

4.1 Choix matériel

Le module de terrain repose sur trois composants, sélectionnés pour leur faible coût, leur disponibilité et leur adéquation aux contraintes « terrain » (matériel simple, réseau variable) :

- un microcontrôleur **NodeMCU v2 (ESP8266, ESP-12E)** : Wi-Fi intégré, logique 3,3 V, largement documenté et suffisant pour la tâche ;
- un capteur **DHT22** : mesure de température (-40 à $+80$ °C) et d’humidité relative (0 à 100 %), précision de l’ordre de $\pm 0,5$ °C ;
- un écran **OLED SSD1306** (0,96", 128×64 , I²C) : affichage local des mesures et des messages d’état, utile en entrepôt.

4.2 Câblage

Le capteur et l’écran sont alimentés en 3,3 V. Le DHT22 utilise une liaison à un fil de données ; l’OLED communique en I²C (deux fils SDA/SCL). Le tableau suivant récapitule les connexions réellement implémentées dans le *firmware*.

Composant	Broche	NodeMCU	GPIO
DHT22	DATA	D4	GPIO2
DHT22	VCC	3V3	—
DHT22	GND	GND	—
OLED SSD1306	SDA	D2	GPIO4
OLED SSD1306	SCL	D1	GPIO5
OLED SSD1306	VCC	3V3	—
OLED SSD1306	GND	GND	—

TABLE 4.1 – Câblage du module IoT (DHT22 + OLED SSD1306 sur NodeMCU ESP8266).

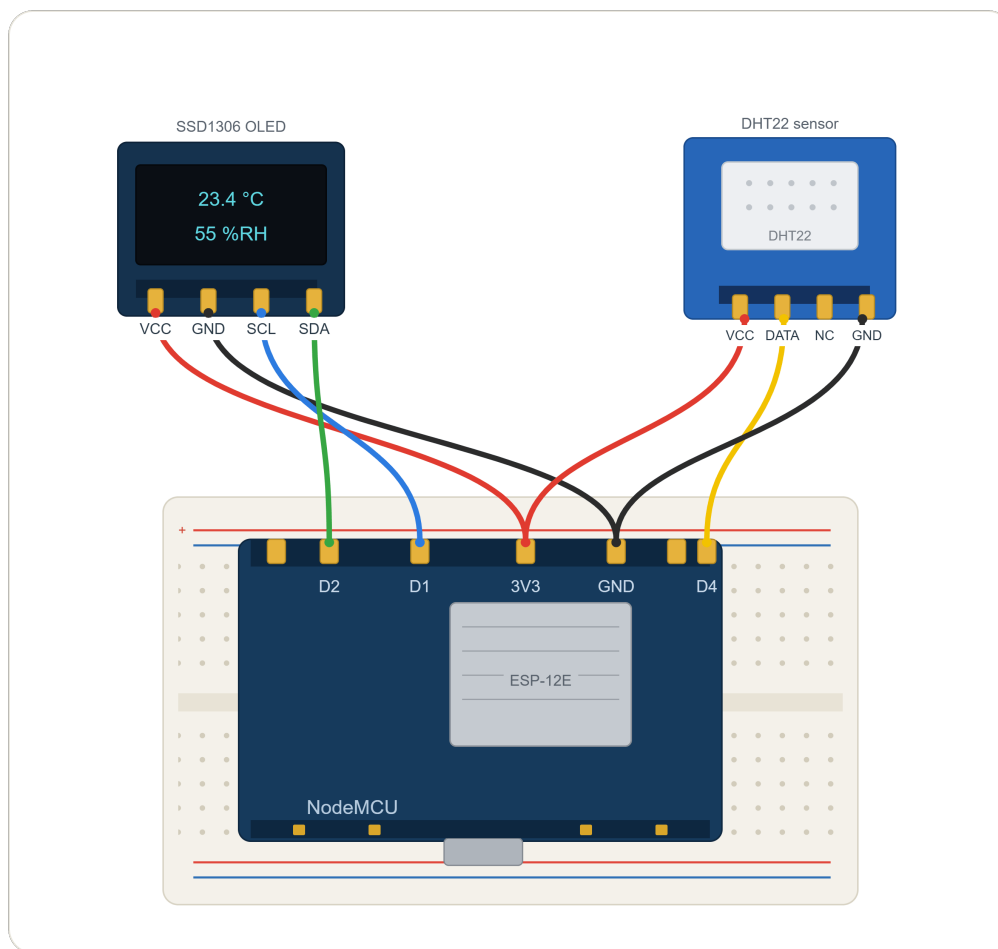


FIGURE 4.1 – Schéma de câblage du module IoT.

4.3 Environnement logiciel

Le *firmware* est développé en C++ (**framework Arduino**) et construit avec **PlatformIO** (carte `nodemcu2`). Il s'appuie sur des bibliothèques éprouvées :

- Adafruit DHT sensor library (+ Adafruit Unified Sensor) : lecture du DHT22 ;
- Adafruit SSD1306 (+ Adafruit GFX) : pilotage de l'écran ;
- PubSubClient : client MQTT ;
- ArduinoJson : sérialisation / désérialisation JSON.

4.4 Protocole MQTT

4.4.1 Topics

Les mesures transitent par le broker MQTT selon une convention hiérarchique. Le back-end pays **s'abonne** au motif suivant (le + est un joker sur l'entrepôt / le capteur) :

`futurekawa/<pays>/+/mesures`

Le capteur **publie** sur un topic concret, par exemple `futurekawa/bresil/entrepot-nord/mesures`. Ce topic n'est pas figé dans le code : il est fourni au capteur par **configuration à distance** (cf. section 4.5).

4.4.2 Format du payload

Chaque mesure est publiée sous forme d'un objet JSON compact :

```
1 {"capteurId": "BR-ENT1-C1", "temperature": 24.3, "humidite": 56.0}
```

Listing 4.1 – Payload d'une mesure publiée en MQTT.

Le champ `capteurId` est optionnel : s'il n'a pas été configuré, le payload ne contient que la température et l'humidité.

4.4.3 Fréquence d'envoi

La période de publication est **paramétrable à distance**. Elle est bornée par un **plancher de 2 secondes** (pour éviter de saturer le réseau et le broker), et une valeur par défaut est définie dans la configuration du *firmware*.

4.5 Configuration et pilotage à distance

Le capteur ne publie *rien* par défaut : il attend d'être configuré et démarré, ce qui évite tout envoi parasite. Il s'abonne pour cela à deux topics de contrôle :

- un **topic de configuration** : il reçoit le contexte de publication (topic cible, identifiant de capteur, fréquence, libellés pays / exploitation / entrepôt) ;
- un **topic de commande** : il reçoit un ordre de démarrage ou d'arrêt de la publication (`{"actif": true|false}`).

Exemple de message de configuration :

```
1 {"topic": "futurekawa/bresil/entrepot-nord/mesures",
2  "capteurId": "BR-ENT1-C1", "frequence": 10,
3  "pays": "Bresil", "exploitation": "Fazenda Sol",
4  "entrepot": "Entrepot Nord", "capteur": "Capteur 1"}
```

Listing 4.2 – Message de configuration reçu par le capteur.

Cette approche découple le *firmware* du déploiement : un même binaire peut servir n'importe quel entrepôt, la cible étant injectée dynamiquement.

4.6 Reconnexion et gestion des erreurs

La robustesse « terrain » est assurée par plusieurs garde-fous :

- **Reconnexion Wi-Fi** : en cas de perte de connexion, le module relance la connexion et attend le rétablissement (avec temporisation), sans bloquer définitivement.
- **Reconnexion MQTT** : à chaque cycle, la connexion au broker est vérifiée et rétablie si nécessaire ; les abonnements de contrôle sont alors restaurés.
- **Lecture capteur invalide** : si le DHT22 renvoie une valeur NaN, la mesure est **ignorée** (pas de publication de donnée erronée) et un message d'erreur est affiché sur l'OLED.
- **Publication conditionnelle** : l'envoi n'a lieu que si la publication est active, le contexte configuré et le topic renseigné — ce qui empêche tout envoi non maîtrisé.
- **Plancher de fréquence** : la période minimale de 2 s protège le broker d'une sur-sollicitation.

4.7 Limites et risques

- **Précision et cadence** du DHT22 limitées (capteur grand public, ~0,5 Hz) : suffisant pour surveiller des dérives lentes, pas pour du contrôle fin.

- **Point de défaillance unique** : un seul capteur par entrepôt dans le prototype ; une redondance serait nécessaire en production.
- **Sécurité MQTT** : le broker est configuré en accès anonyme pour la démonstration ; en production, il faudrait authentification et chiffrement (TLS).
- **Absence de tampon local** : en cas de coupure réseau prolongée, les mesures ne sont pas mémorisées localement (pas de stockage sur le microcontrôleur).
- **Dépendance au Wi-Fi** : les entrepôts à couverture réseau faible nécessiteraient une solution de repli (LoRa, stockage différé...).

Troisième partie

Qualité et industrialisation

Chapitre 5

Plans de tests détaillés

5.1 Stratégie de test

La stratégie privilégie des **tests unitaires automatisés** ciblant la **logique métier critique** : le moteur d’alertes (seuils, hystérésis, péremption), le contrôle d’accès par pays, la consolidation multi-pays et les fonctions d’affichage du front. Ces briques concentrent le risque fonctionnel ; les sécuriser par des tests reproductibles garantit la non-régression à chaque évolution.

Les tests sont **isolés** : ils ne requièrent ni base de données, ni broker MQTT, ni serveur démarré. Les dépendances externes (dépôts, service de notification, clients REST) sont **simulées** (Mockito). Ils sont donc rapides, déterministes et rejouables à l’identique en local comme en intégration continue.

Périmètre	Type	Outils	Nombre
Back-end pays	Unitaire (métier)	JUnit 5 + Mockito	16
Back-end central	Unitaire (métier)	JUnit 5 + Mockito	6
Front-end	Unitaire (UI/helpers)	Vitest	9
Total			31

TABLE 5.1 – Typologie et répartition des tests automatisés.

Ces tests sont **lançables manuellement** (`./mvnw test` pour les back-ends, `npm test` pour le front) et **rejoués automatiquement** par la chaîne d’intégration continue, qui publie les résultats au format JUnit.

5.2 Cas de test

5.2.1 Back-end pays — moteur d’alertes

Cas	Donnée d’entrée	Résultat attendu
Anti-flapping	1 mesure hors plage	aucune alerte levée
Ouverture confirmée	3 mesures consécutives hors plage	exactement 1 alerte + 1 e-mail
Péréemption	lot stocké depuis > 365 jours	lot PERIME + alerte
Anti-doublon péréemption	alerte péréemption déjà active	pas de seconde alerte
Hystérésis — cumul anomalies	mesures anormales successives	compteur d’anomalies incrémenté
Hystérésis — cumul normales	mesures normales successives	compteur de normales incrémenté
Hystérésis — reset anomalies	mesure normale après anomalies	compteur d’anomalies remis à 0
Hystérésis — reset normales	mesure hors plage après normales	compteur de normales remis à 0
Hystérésis — isolation	deux entrepôts distincts	compteurs indépendants
Seuils — bornes	idéal $\pm$ tolérance	bornes min/max correctes
Seuils — valeur idéale	valeur centrale	dans la bande
Seuils — bornes incluses	valeur = borne	dans la bande ($\geq$ et $\leq$)
Seuils — température trop haute	temp > max	hors bande
Seuils — température trop basse	temp < min	hors bande
Seuils — humidité hors plage	humidité hors, temp correcte	hors bande
Seuils — tout hors plage	temp et humidité hors	hors bande

TABLE 5.2 – Cas de test du back-end pays (16 tests).

5.2.2 Back-end central — isolation et consolidation

Cas	Donnée d’entrée	Résultat attendu
Super admin	rôle « tous pays » (*)	accès à n’importe quel pays
Admin de son pays	admin BR interrogeant BR (casse ignorée)	accès autorisé
Admin d’un autre pays	admin BR interrogeant EC	accès refusé (403)
Fusion multi-pays	lots de deux pays	lots fusionnés et tagués du code pays
Tolérance aux pannes	un pays injoignable	pays ignoré, les autres renvoyés
Pays inconnu	code pays invalide	erreur 404

TABLE 5.3 – Cas de test du back-end central (6 tests).

5.2.3 Front-end — fonctions d’affichage utilisateur

Cas	Donnée d’entrée	Résultat attendu
Initiales (prénom+nom)	« Jean Dupont »	« JD »
Initiales (repli e-mail)	e-mail sans nom	première lettre de l’e-mail
Initiales (défaut)	aucune information	« U »
Nom affiché	prénom + nom	« Jean Dupont »
Nom affiché (repli)	prénom/nom absents	e-mail utilisé
Nom affiché (nul)	utilisateur nul	chaîne vide
Libellé rôle (super admin)	rôle « tous pays »	« siège / tous pays »
Libellé rôle (admin pays)	code pays	libellé du pays traduit
Libellé rôle (inconnu)	rôle non reconnu	« Lecteur » par défaut

TABLE 5.4 – Cas de test du front-end (9 tests).

5.3 Données de test

Les jeux de données sont **construits dans les tests eux-mêmes** (pas de base externe) : mesures de température / humidité choisies pour tomber juste à l’intérieur ou juste à l’extérieur de la bande, lots horodatés au-delà ou en deçà du seuil de 365 jours, utilisateurs porteurs de différents rôles (super admin, admin pays, rôle inconnu). Cette maîtrise des entrées rend chaque résultat **prévisible et vérifiable**.

5.4 Critères de réussite

- chaque cas de test valide ses assertions (résultat obtenu = résultat attendu) ;
- la suite complète (**31/31**) doit être verte pour qu’un *build* soit considéré comme réussi ;
- les résultats sont publiés au format **JUnit** et consultables dans l’interface d’intégration continue.

5.5 Gestion des anomalies

Le traitement d’une anomalie suit un cycle simple et outillé :

1. **Constat** : un test échoue (ou un dysfonctionnement est signalé) ; la trace et les logs localisent la cause.
2. **Correction** : le code fautif est corrigé, si besoin en ajoutant un nouveau cas de test qui reproduit le défaut.
3. **Re-test** : la suite est rejouée localement pour confirmer la correction.
4. **Non-régression** : à chaque *commit*, la chaîne d’intégration continue rejoue l’intégralité des tests, empêchant la réapparition du défaut.

À titre d’illustration, le cas « anti-flapping » garantit qu’une variation passagère ne déclenche pas de fausse alerte : si une évolution du moteur cassait cette règle, le test correspondant repasserait au rouge et bloquerait la livraison.

5.6 Limites et perspectives

Les tests actuels couvrent la **logique métier** au niveau unitaire. Pour aller plus loin, on ajouterait des **tests d’intégration** (API REST + base réelle via conteneurs éphémères), des **tests d’API** de bout en bout (contrats REST) et des **tests d’interface** automatisés (parcours utilisateur), afin de compléter la pyramide de tests.

Chapitre 6

Industrialisation : intégration et déploiement continu

6.1 Objectifs et démarche

Pour garantir un niveau de qualité « projet » et préparer une mise en production progressive, la solution est outillée d’une chaîne **CI/CD** (intégration et déploiement continu). À chaque évolution du code, cette chaîne **rejoue automatiquement** une séquence d’étapes — compilation, tests, packaging, construction d’images — puis produit des artefacts prêts à être déployés. Deux niveaux se distinguent :

- l’**intégration continue (CI)** : compiler, tester et empaqueter à chaque modification, afin de détecter au plus tôt toute régression ;
- la **livraison / déploiement continu (CD)** : publier les images sur un registre puis les déployer sur un environnement d’exécution.

L’outil retenu est **Jenkins**, piloté par un pipeline **défini en code** (fichier **Jenkinsfile** versionné dans le dépôt) : le processus est ainsi traçable, revu et reproductible au même titre que le code applicatif.

6.2 Une chaîne reproductible

Pour qu’elle soit jouable sur n’importe quel poste, la chaîne est elle-même **conteneurisée** : une image Docker dérivée de *Jenkins LTS* embarque **JDK 21**, **Node 20**, le **client Docker** et les extensions nécessaires ; le *job* d’intégration est pré-configuré au démarrage. Le dépôt est monté dans le conteneur et le pipeline y lit le **Jenkinsfile**. Une simple construction d’image suivie d’un lancement suffit donc à disposer d’un serveur d’intégration opérationnel.

6.3 Le pipeline d’intégration continue

La partie CI enchaîne cinq étapes, exécutées à chaque *build* :

Étape	Action
Checkout	Récupération du code depuis le dépôt Git.
Tests	Exécution des 31 tests automatisés (back-end pays, back-end central, front-end) et publication des résultats au format JUnit.
Packaging	Construction des archives JAR des back-ends et du <i>build</i> de production du front-end.
Images Docker	Construction des images conteneurisées des deux back-ends.
Archivage	Publication des artefacts (JAR , <i>build</i> front) téléchargeables depuis Jenkins.

TABLE 6.1 – Étapes de la chaîne d'intégration continue.

En cas d'échec d'une étape, le *build* est marqué en erreur et les artefacts ne sont pas publiés : un code non conforme ne peut donc pas être livré.

6.4 Packaging et construction des images

Une attention particulière a été portée à l'**efficacité** de l'étape de construction des images. Plutôt que de recompiler l'application à l'intérieur de l'image (ce qui retéléchargeait toutes les dépendances et allongeait fortement le *build*), un **Dockerfile** dédié à la CI **réutilise l'archive JAR déjà produite** à l'étape de packaging. Cette optimisation a ramené la construction des images de plusieurs minutes à quelques secondes, illustrant le principe « ne pas refaire le travail déjà fait ».

Chaque image est étiquetée avec le **numéro de build** (traçabilité) et l'étiquette **latest** (dernière version) :

```

1 docker build -f backend-pays/Dockerfile.ci \
2   -t futurekawa/backend-pays:$BUILD_NUMBER \
3   -t futurekawa/backend-pays:latest backend-pays

```

Listing 6.1 – Construction des images à partir des artefacts.

6.5 La livraison continue

Au-delà de l'intégration, le pipeline décrit une **chaîne de déploiement complète**, déclenchée uniquement sur la branche principale :

1. **Publication au registre** : les images sont poussées sur un registre Docker (*Docker Hub*) via des identifiants sécurisés stockés dans Jenkins ;
2. **Déploiement de test** : les images sont déployées sur un cluster **Kubernetes** d'environnement **TEST** ;
3. **Approbation** : le pipeline se met en pause et attend la validation **manuelle** d'un superviseur habilité (porte de contrôle) ;
4. **Déploiement de production** : après validation, les images sont déployées sur le cluster **PROD**.

Cette porte d'approbation humaine entre **TEST** et **PROD** matérialise un compromis maîtrisé entre automatisation et contrôle : la production n'est jamais touchée sans décision explicite.

6.6 Le déploiement Kubernetes

Le déploiement s'appuie sur un **manifeste Kubernetes** paramétrable qui décrit l'ensemble du système : broker MQTT, back-end central, back-ends pays, ainsi que les services réseau associés. Ce manifeste met en œuvre plusieurs mécanismes de résilience :

- des **sondes de vivacité et de disponibilité** (*liveness / readiness*) qui redémarrent automatiquement un conteneur défaillant ;
- une **réplication** du back-end central (plusieurs instances) ;
- l'externalisation de la configuration et des secrets (*ConfigMap* et *Secret*), séparant le code des paramètres sensibles ;
- des **mise à jour progressives** (*rolling updates*) sans interruption de service.

6.7 Preuves d'exécution

La chaîne a été exécutée et vérifiée de bout en bout.

Intégration continue. Un *build* complet passe au vert : les 31 tests réussissent, les artefacts sont archivés et les images sont construites (figure 6.1).

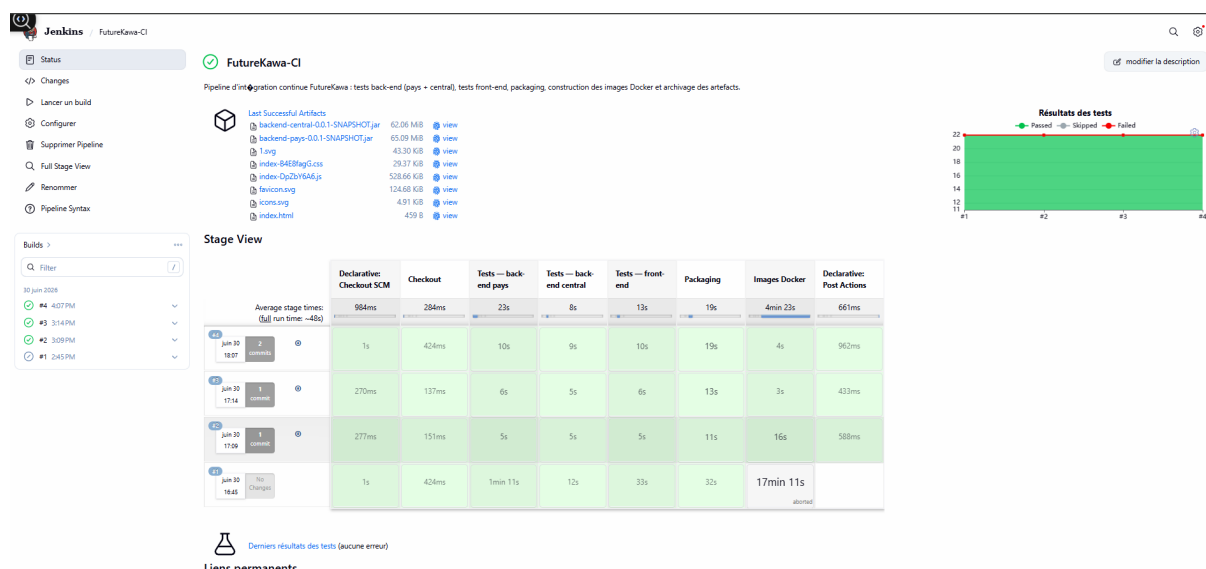


FIGURE 6.1 – Vue par étapes d'un *build* Jenkins réussi (tests, packaging, images).

Déploiement. Le manifeste a été appliqué avec succès sur un cluster Kubernetes : l'ensemble des services (base de données, broker MQTT, back-end central et back-ends pays) atteint l'état **Running** (figure 6.2). Les redémarrages automatiques observés lors d'un redémarrage du poste illustrent concrètement l'**auto-réparation** du dispositif.

```
(base) PS C:\Users\ian_chel\Documents\GitHub\FutureKawa> kubectl get pods -n futurekawa-test
```

NAME	READY	STATUS	RESTARTS	AGE
fk-central-77fc4d895f-xz8r9	1/1	Running	4 (86m ago)	17h
fk-mosquitto-79c7b5fccd-djzmf	1/1	Running	5 (86m ago)	20h
fk-pays-br-766d646c4b-tz9wm	1/1	Running	3 (86m ago)	17h
fk-pays-co-7f6d48d77f-sfnht	1/1	Running	3 (86m ago)	17h
fk-pays-ec-6d6679f876-wkgct	1/1	Running	3 (86m ago)	17h
fk-postgres-57bdbcf94f-fw26	1/1	Running	5 (86m ago)	20h

FIGURE 6.2 – Déploiement Kubernetes : tous les services à l'état **Running**.

6.8 Limites et perspectives

Le déploiement a été **prouvé sur un cluster Kubernetes local** ; le branchement sur une infrastructure *cloud* de production (par exemple Azure AKS) est **prêt** au niveau du pipeline et des manifestes, mais non provisionné dans le cadre du prototype (coût). Les évolutions naturelles sont : le déclenchement automatique du pipeline sur *push* (*webhook*), l'ajout d'une étape d'analyse de qualité de code, et la sécurisation du registre et du broker (authentification, chiffrement).

Quatrième partie

Déploiement, accompagnement et bilan

Chapitre 7

Documentation utilisateur

7.1 Objet et public visé

Ce guide s'adresse aux **utilisateurs métier** de FutureKawa : responsables d'exploitation et d'entrepôt, référents qualité, équipes Supply Chain et siège. Il présente l'interface web (« Console d'administration ») et décrit, pas à pas, les tâches courantes : consulter les stocks, lire les mesures, traiter les alertes et, pour les administrateurs, gérer les comptes et les paramètres. Aucune connaissance technique n'est requise.

7.2 Connexion et principes généraux

L'accès se fait depuis un navigateur, avec une **adresse e-mail** et un **mot de passe**. Une fois connecté, l'écran se compose de trois zones :

- la **barre supérieure** : un *sélecteur de pays* (Tous / Brésil / Équateur / Colombie) qui filtre tout l'affichage, l'identité de l'utilisateur, son rôle et le bouton de déconnexion ;
- le **menu de gauche**, séparé en *Navigation* (consultation) et *Administration* (gestion, visible selon les droits) ;
- la **zone centrale**, qui affiche la page sélectionnée.

Le **périmètre dépend du rôle** : un *super-administrateur* (siège) voit l'ensemble des pays, tandis qu'un *administrateur pays* ne voit que le sien — le sélecteur de pays est alors restreint.

7.3 Le tableau de bord

Le tableau de bord offre une **vue consolidée** de l'état des stocks et des conditions en temps réel. En haut, quatre indicateurs clés : pays connectés, lots en stock, alertes actives et entrepôts suivis. En dessous, la courbe d'**évolution des conditions** et la **répartition des alertes**.

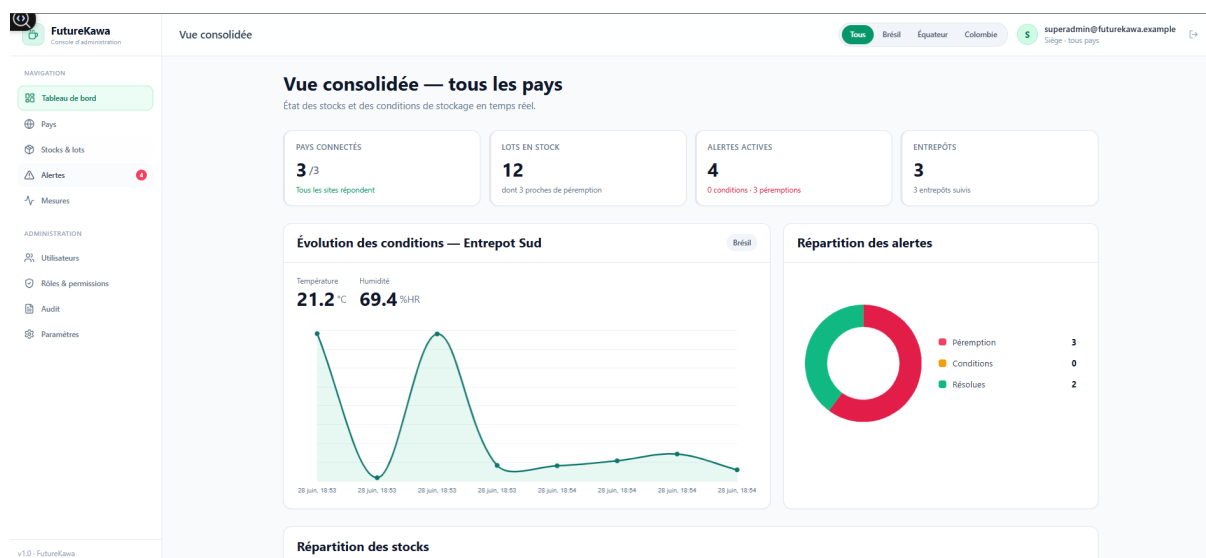


FIGURE 7.1 – Tableau de bord — indicateurs, évolution des conditions et répartition des alertes.

Plus bas, la section « **Alertes à traiter** » propose un accès rapide pour acquitter ou résoudre les incidents, et la « **Synthèse par pays** » récapitule, pour chaque pays, le nombre de lots, les alertes actives, les capteurs, la température moyenne et le statut. Le bouton **Exporter** permet d'extraire cette synthèse.

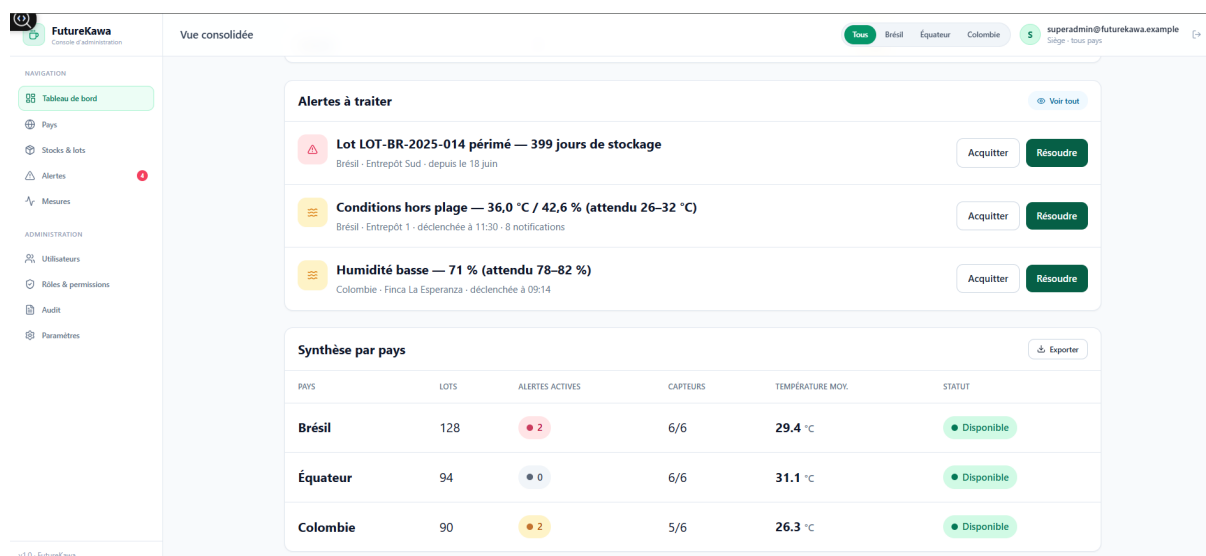


FIGURE 7.2 – Tableau de bord — alertes à traiter et synthèse par pays.

7.4 Consulter les stocks et les lots

La page **Stocks & lots** liste tous les lots de café et leur état qualité. Quatre compteurs résument la situation : total, conformes, en alerte et périmés.

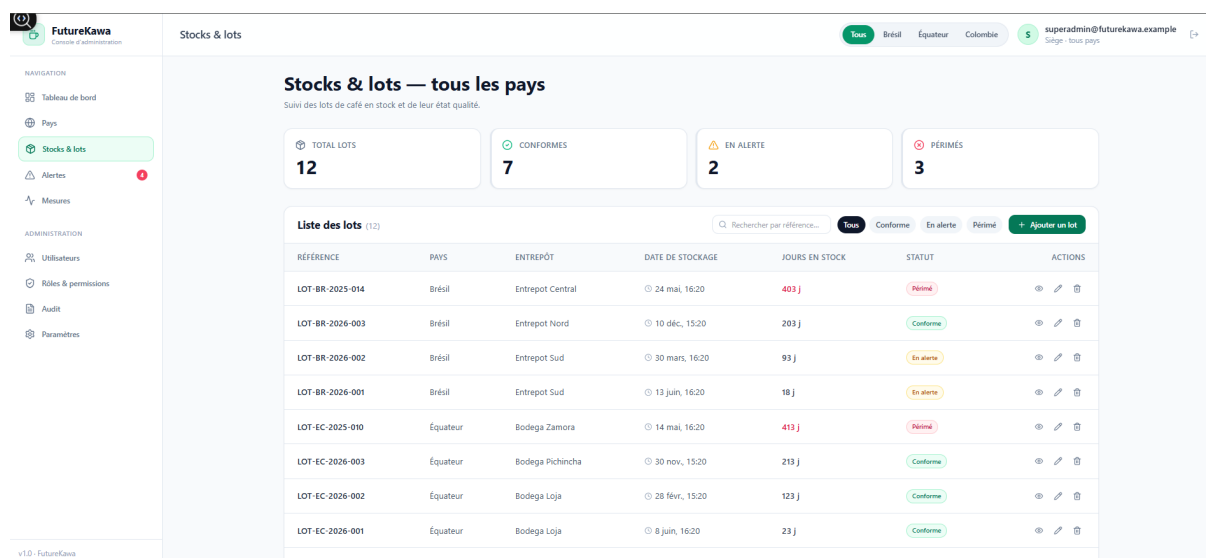


FIGURE 7.3 – Stocks & lots — liste des lots triés par date de stockage.

Chaque lot affiche sa référence, son pays, son entrepôt, sa date de stockage, son ancienneté (*jours en stock*) et son **statut** :

- **Conforme** : conditions dans la plage acceptable ;
- **En alerte** : conditions hors plage dans l'entrepôt du lot ;
- **Périmé** : lot stocké depuis plus de 365 jours.

La liste est **triée par date** (logique FIFO : les plus anciens en premier) et peut être filtrée par statut ou recherchée par référence. Pour chaque lot, les actions permettent de le **consulter**, le **modifier** ou le **supprimer**. Le bouton « + **Ajouter un lot** » ouvre le formulaire de création (référence, pays, exploitation, entrepôt, date de stockage).

7.5 Lire les mesures (courbes)

La page **Mesures** présente l'historique des conditions relevées par les capteurs. On sélectionne un **entrepôt** et une **période** (24 h, 7 jours ou 30 jours), puis deux courbes s'affichent : **température** et **humidité**, avec la dernière valeur mise en évidence.

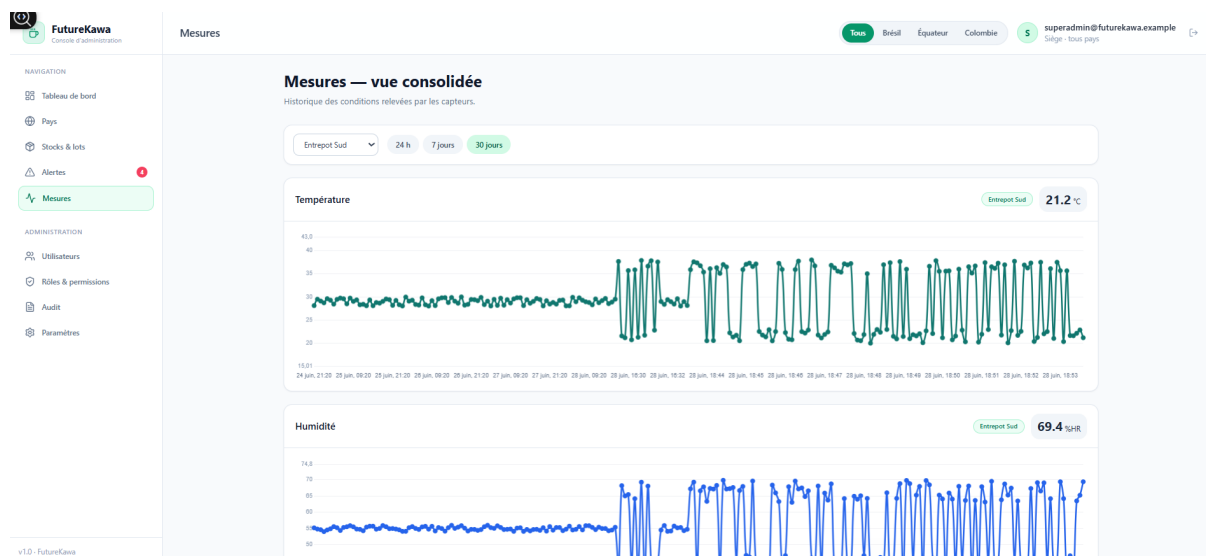


FIGURE 7.4 – Mesures — courbes de température et d'humidité par entrepôt.

Ces courbes permettent d'**analyser les dérives** et de **justifier la traçabilité** des conditions de stockage auprès des clients.

7.6 Comprendre et traiter les alertes

La page **Alertes** recense les incidents. Deux **types** existent : **Conditions** (température ou humidité hors plage) et **Péréemption** (lot de plus de 365 jours). Les alertes sont réparties en trois onglets : **Actives**, **Acquittées** et **Résolues**.

TYPE	DESCRIPTION	ENTREPÔT	DÉCLENCHÉE	NOTIFS	ACTIONS
Conditions	Conditions hors plage dans l'entrepôt 1 : 37.4°C / 43.0% (attendu 26-32°C / 53-57%)	#1	28 juin, 18:44	71	Acquitter Résoudre
Péréemption	Lot LOT-BR-2025-014 périmé : 400 jours de stockage (> 365)	#3	28 juin, 16:21	0	Acquitter Résoudre
Péréemption	Lot LOT-EC-2025-010 périmé : 410 jours de stockage (> 365)	#2	28 juin, 16:21	0	Acquitter Résoudre
Péréemption	Lot LOT-CO-2025-007 périmé : 420 jours de stockage (> 365)	#2	28 juin, 16:21	0	Acquitter Résoudre

FIGURE 7.5 – Alertes actives — type, description, entrepôt et actions.

Chaque alerte indique la valeur mesurée, la plage attendue, l'entrepôt concerné, la date de déclenchement et le nombre de notifications envoyées. Deux actions sont possibles :

- **Acquitter** : signaler que l'incident est pris en charge ;
- **Résoudre** : clôturer l'incident une fois traité.

Lorsqu'une alerte de conditions se déclenche, un **e-mail** est envoyé au responsable du pays concerné. Si les conditions reviennent durablement dans la plage, l'alerte est **résolue automatiquement** (mode **AUTO**), comme le montre l'historique des alertes résolues.

Historique des alertes
Incidents clôturés, avec durée et mode de résolution.

Actives (4) Acquittées (0) Résolues (2)

TYPE	DESCRIPTION	ENTREPÔT	DÉCLANCHÉE	RÉSOLUE	MODE
Conditions	Conditions hors plage dans l'entrepôt 3 : 22.9°C / 42.0% (attendu 26-32°C / 53-57%)	#3	28 juin, 18:47	28 juin, 18:47	AUTO
Conditions	Conditions hors plage dans l'entrepôt 1 : 21.2°C / 65.0% (attendu 26-32°C / 53-57%)	#1	28 juin, 16:30	28 juin, 16:32	AUTO

FIGURE 7.6 – Historique des alertes résolues (avec mode et durée de résolution).

7.7 Fonctions d'administration

Les fonctions suivantes sont réservées aux profils *super-administrateur* et *administrateur pays*.

7.7.1 Gestion des utilisateurs

La page **Utilisateurs** permet de créer, activer, désactiver ou modifier les comptes, chacun associé à un **rôle** et à un **pays**. Recherche et filtres par rôle facilitent la gestion.

Utilisateurs
Gérez les comptes et leurs accès selon votre périmètre.

Rechercher un nom ou un e-mail...

Tous les rôles Admin pays Opérateur Lecteur + Nouvel utilisateur

UTILISATEUR	RÔLE	PAYS	DERNIÈRE CONNEXION	ACTIF
AB Admin Brésil admin-bresil@futurekawa.example	Admin Pays	Brésil	—	☒
AE Admin Équateur admin-equateur@futurekawa.example	Admin Pays	Équateur	—	☒
AC Admin Colombie admin-colombie@futurekawa.example	Admin Pays	Colombie	—	☒
SC Sofiane Chelghoum sofchel2000@gmail.com	Admin Pays	Brésil	29 juin, 16:42	☒
ST Simulateur Technique simulateur@futurekawa.example	Lecteur	Tous	29 juin, 17:33	☒
SS Super Siège superadmin@futurekawa.example	Super Admin	Tous	1 juil., 20:35	☒

FIGURE 7.7 – Gestion des utilisateurs et de leurs accès.

7.7.2 Rôles et permissions

La page **Rôles & permissions** présente, sous forme de matrice, les droits accordés à chaque rôle (Super Admin, Admin Pays, Opérateur, Lecteur). Le périmètre pays s'applique en plus de ces permissions.

Permission	Super Admin	Admin Pays	Opérateur	Lecteur
Consulter stocks & mesures	✓	✓	✓	✓
Enregistrer un lot	✓	✓	✓	—
Acquitter / résoudre une alerte	✓	✓	✓	—
Gérer les utilisateurs	✓	✓	—	—
Supprimer un compte	✓	—	—	—
Accéder à tous les pays	✓	—	—	—
Consulter le journal d'audit	✓	✓	—	—

■ En lecture seule dans cette version. L'édition des permissions sera ajoutée plus tard.

FIGURE 7.8 – Matrice des permissions par rôle.

7.7.3 Paramètres système

La page **Paramètres** regroupe, **par pays**, les **seuils de conformité** (température et humidité cibles, tolérances) et le **comportement des alertes** (mesures consécutives avant ouverture et résolution, intervalle de rappel, seuil de préemption, notifications par e-mail).

Paramètres système
Seuils de conformité et comportement des alertes, par pays.

Brésil Équateur Colombie

Seuils de conformité — Brésil

Température cible (°C): 29 Tolérance température (± °C): 3

Humidité cible (%): 55 Tolérance humidité (± %): 2

Comportement des alertes

Mesures consécutives avant ouverture: 3 Mesures consécutives avant résolution: 3

Intervalle de rappel (minutes): 30 Seuil de préemption (jours): 365

Notifications par e-mail
Envoyer un e-mail au responsable du pays: ☒

Réinitialiser Enregistrer

■ Aujourd'hui ces valeurs vivent dans les fichiers de configuration. Les rendre éditables nécessite un endpoint backend.

FIGURE 7.9 – Paramètres système — seuils et comportement des alertes par pays.

7.7.4 Journal d'audit

La page **Audit** trace les actions sensibles (qui a fait quoi, quand, sur quel pays et avec quel résultat), au service de la traçabilité et de la sécurité.

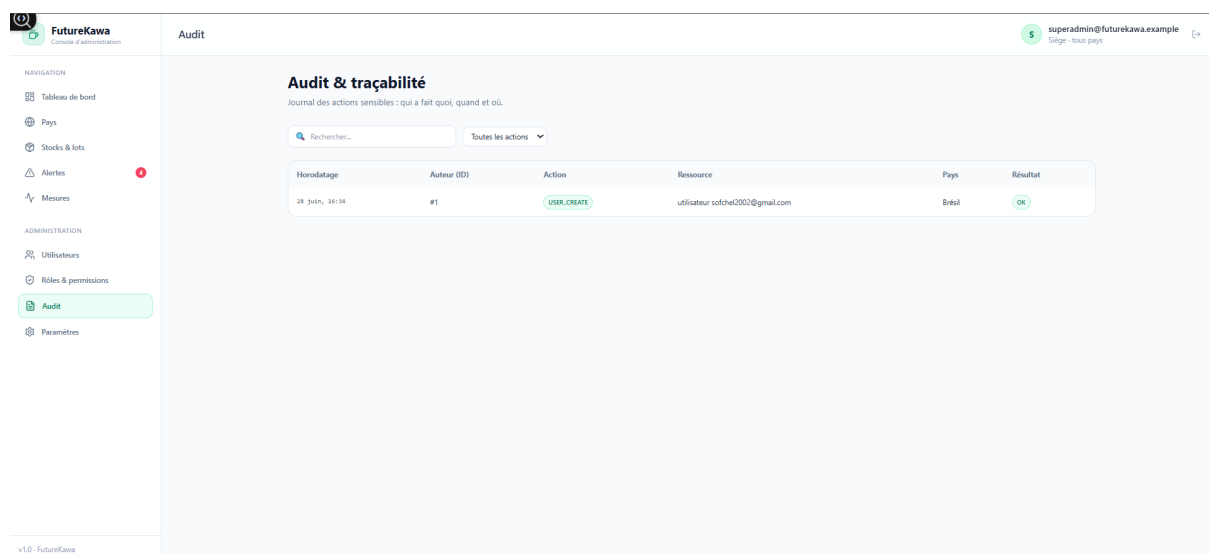


FIGURE 7.10 – Journal d’audit et de traçabilité.

7.8 FAQ — résolution des problèmes courants

- **Je ne vois qu’un seul pays.** C’est normal si votre rôle est « administrateur pays » : votre périmètre est limité à votre pays.
- **Une alerte revient régulièrement.** Tant qu’un incident est actif, un rappel est envoyé à intervalle régulier. Résolvez-le, ou attendez le retour durable des conditions dans la plage (résolution automatique).
- **Un lot passe « En alerte ».** Les conditions sont hors plage dans son entrepôt : consultez la page **Mesures** pour analyser la dérive.
- **Je n’ai pas reçu d’e-mail d’alerte.** En environnement de démonstration, les e-mails sont capturés par un outil de test (Mailpit) et ne sont pas envoyés à de vraies adresses.
- **Je ne peux pas modifier les seuils.** Dans cette version, les paramètres sont en lecture seule ; leur édition sera ajoutée ultérieurement.
- **Ma session s’est terminée.** Reconnectez-vous : le jeton d’accès a une durée limitée et est renouvelé automatiquement en cours d’utilisation.

Chapitre 8

Conduite du changement

8.1 Justification de la démarche

L'intégration de la solution FutureKawa dans les habitudes de travail des équipes constitue un changement organisationnel qui peut susciter des **réticences**, notamment liées à la perte de repères par rapport aux outils existants (tableurs, relevés manuels) et au doute sur la fiabilité d'un système automatisé. Il est donc nécessaire de mettre en place une démarche structurée d'accompagnement afin de garantir une **adoption réussie** de l'outil.

Cette démarche s'appuie sur le **modèle de transition de William Bridges**, qui décompose le changement en trois phases : la *fin* de l'ancien mode de fonctionnement (deuil des anciens outils), la *zone neutre* (période d'adaptation et d'apprentissage) et le *nouveau départ* (adoption complète du nouvel outil).

8.2 Plan d'action sur les quatre axes

Informier. En amont du déploiement, une réunion de présentation est organisée avec les responsables d'exploitation, d'entrepôt et les référents qualité afin d'expliquer les objectifs du projet, les bénéfices attendus (gain de temps, fiabilité des données, traçabilité) et le calendrier de mise en œuvre. Une note de cadrage et une FAQ répondent aux inquiétudes (rôles, surveillance, fiabilité).

Communiquer. Un canal de communication dédié (lettre d'information, affichage en entrepôt) est mis en place pendant toute la durée du déploiement pour diffuser les points d'avancement, recueillir les retours du terrain et valoriser les premières réussites.

Former. Des sessions de formation pratiques, adaptées à chaque profil (entrepôt, qualité, siège), sont proposées aux futurs utilisateurs, accompagnées d'un guide utilisateur détaillant la prise en main de l'application (consultation des stocks, lecture des courbes, traitement des alertes). Les correspondants SI locaux sont formés comme référents et assurent un support de proximité.

Faire participer. Une phase pilote est mise en place sur un premier site avant le déploiement généralisé. Des ambassadeurs sont désignés parmi les profils moteurs, et les retours de la phase pilote sont collectés et intégrés pour ajuster l'outil et les procédures avant la généralisation.

8.3 Mise en application

Le déploiement est **progressif** : un site pilote valide l'outil et l'accompagnement en conditions réelles, avant une généralisation par pays. L'adoption est suivie à l'aide d'indicateurs simples (taux d'utilisateurs actifs, abandon des tableurs, sollicitations du support, satisfaction). Le plan

est formalisé par un **support de présentation** destiné aux équipes et un **guide utilisateur** synthétique. Cette approche progressive, associant information, formation et implication des utilisateurs, vise à limiter les résistances au changement et à favoriser une adoption durable de la solution.

Chapitre 9

Conclusion

Le projet FutureKawa avait pour objectif de doter l'entreprise d'un outil unique et fiable pour **suivre les stocks** de café vert et **surveiller automatiquement les conditions de conservation** dans les entrepôts de ses trois pays, tout en préparant une évolution vers l'automatisation. Le prototype livré répond à cet objectif.

9.1 Bilan du travail réalisé

La solution mise en œuvre couvre l'ensemble de la chaîne, du capteur à la consultation :

- une **architecture distribuée** pays $\leftrightarrow$ siège, avec un back-end autonome et une base dédiée par pays, et un back-end central de consolidation ;
- un **module IoT** (ESP8266 + DHT22 + OLED) qui relève température et humidité et les publie en MQTT, avec configuration et pilotage à distance ;
- un **moteur d'alertes** automatique (conditions hors plage avec hystérésis, péremption au-delà de 365 jours) doublé d'une notification par e-mail ;
- un **front-end web** de consultation des stocks, des courbes de mesures et des alertes ;
- une démarche de **qualité « projet »** : 31 tests automatisés, une chaîne d'intégration continue et une exécution conteneurisée (Docker, Kubernetes).

9.2 Conformité aux besoins

Les exigences du cahier des charges sont satisfaites : centralisation du suivi par pays et par entrepôt, traçabilité des lots, surveillance automatisée des conditions, détection et signalement des situations à risque (dérives et lots trop anciens), et mise en place d'une base technique préparant la phase 2. Les choix d'architecture (microservices, isolation par pays, MQTT, REST) ont été justifiés au regard des critères de stabilité, d'efficacité et de tolérance aux pannes.

9.3 Limites

Le périmètre reste celui d'un **prototype avancé**. Les principales limites, assumées, sont : un **unique capteur par entrepôt** (pas de redondance), un broker MQTT en **accès non chiffré** adapté à la démonstration, des tests concentrés au niveau **unitaire**, et un déploiement orchestré **prouvé localement** sur Kubernetes mais non provisionné sur une infrastructure de production.

9.4 Perspectives

La suite naturelle est la **phase 2** : l'automatisation des équipements d'entrepôt (chauffage, humidification, aération) pilotés par les mesures, en s'appuyant sur la chaîne « capteurs $\rightarrow$ dé-

cision → actionneurs » déjà esquissée. D'autres pistes consolideraient la solution : sécurisation du transport MQTT (authentification, TLS), redondance des capteurs, enrichissement de la pyramide de tests (intégration, API, bout en bout) et déploiement continu sur un cluster de production. Le prototype constitue ainsi une **base solide et industrialisable** pour la montée en charge du dispositif chez FutureKawa.